



COEP TECHNOLOGICAL UNIVERSITY
(COEP TECH)
(Formerly College of Engineering Pune)

T2 Examination

(CT-20014) Data Structures and Algorithms – II

Course: B.Tech, Semester IV

Branch: Computer Engineering

Academic Year: 2023-2024

Max.Marks:20

Duration: 1 Hours

Date: 19/03/2024

Instructions:

Student MIS No.

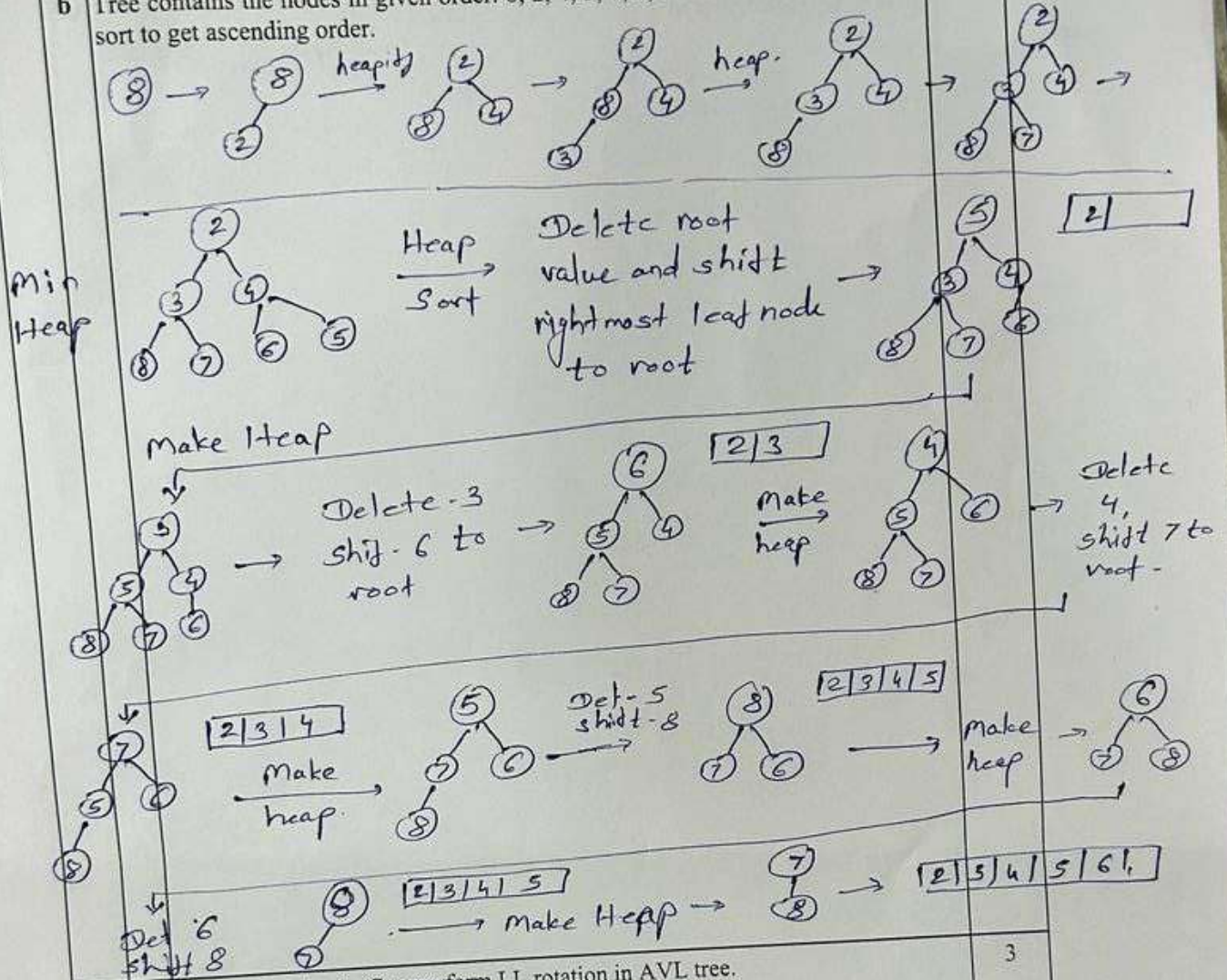
--	--	--	--	--	--	--	--	--	--

- Figures to the right indicate the full marks.
- Mobile phones and programmable calculators are strictly prohibited.
- Writing anything on question paper is not allowed.
- Exchange/Sharing of stationery, calculator etc. not allowed.
- Write your MIS Number on Question Paper

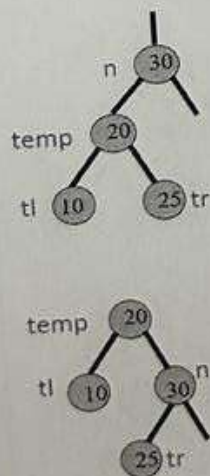
			Marks
Q 1	a	How many maximum and minimum nodes will be there for AVL tree of height 3? Ans: Maximum 15 and minimum 7 Max nodes: $2^{h+1} - 1 = 2^{3+1} - 1 = 15$ Min nodes: $N(h) = N(h-1) + N(h-2) + 1$ $N(3) = N(2) + N(1) + 1$ $= 4 + 2 + 1 = 7$	2
	b	In given graph, find out in-degree and out-degree of 2. Ans: 2 and 3	2
	c	i) What is the advantage of an AVL tree over a BST? ii) Complete graph has _____ number of edges. Ans: i) ii) $n(n-1)/2$	2

→ In BST time complexity depends on height of tree.
- If BST is skewed then time complexity will be $O(n)$
- Whereas AVL is height balanced tree, requires $O(\log n)$

b Tree contains the nodes in given order: 8, 2, 4, 3, 7, 6, 5. Show working of Heap sort to get ascending order.



Q3 a Write a function in C to perform LL rotation in AVL tree.



LL Rotation

```
void LL(avl *t, node *n) {
    node *temp = n->left;
    n->left = temp->right;
    if(n->left) n->left->parent = n;
    temp->right = n;
    temp->parent = n->parent;
    n->parent = temp;
    if(temp->parent) {
        if(temp->parent->left == n) {
            temp->parent->left = temp;
        }
        else {
            temp->parent->right = temp;
        }
    }
    else *t = temp;
    adjustBF(*n);
    adjustBF(*temp);
}
```

b Write a function in C to insert element in a max heap using array.

3

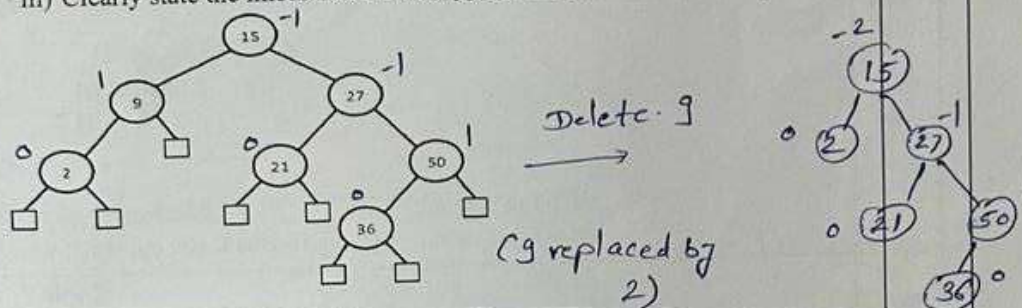
Insert Function

```
void insert(Heap* h, int value) {  
    if (h->rear == h->size - 1) {  
        printf("Heap is full. Cannot insert value.\n");  
        return;  
    }  
  
    h->rear++;  
    int i = h->rear;  
  
    while (i > 0 && value > h->A[(i - 1) / 2]) {  
        h->A[i] = h->A[(i - 1) / 2];  
        i = (i - 1) / 2;  
    }  
  
    h->A[i] = value;  
}
```


Q 2 a

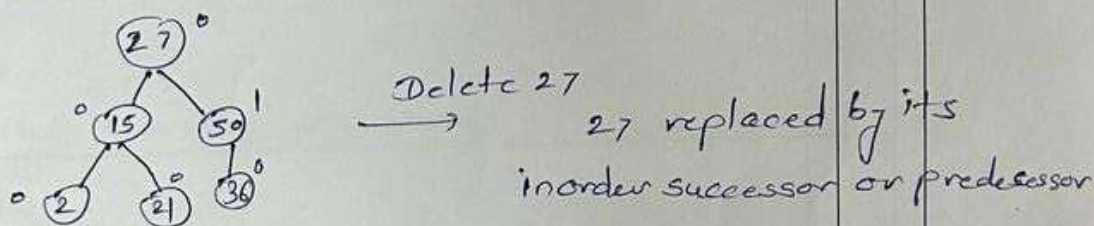
- Show the AVL tree that results after deletion of the integer keys 9 and 27.
- Explain process to re-balance after deletion of the given keys.
- Clearly state the imbalanced node, type of imbalance after the operation.

4

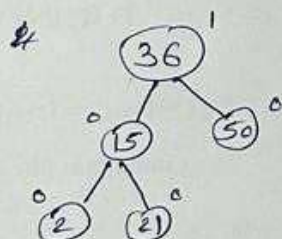


① This is L-1 imbalance. ② Imbalance at 15 is of -2

③ Perform RR rotation

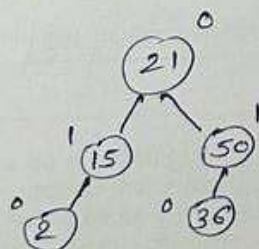


④



No imbalance.
No Rotation.
Final Tree.

OR



No imbalance
No Rotation
Final Tree.